Даня
@ka3ak_0ff

КОНСПЕКТ

Даня
@ka3ak_0ff

Многие наверняка задавались вопросом, а как же легко и без проблем в автоматическом формате можно было бы решать задания из блока теория игр на экзамене на полные 3 из 3 баллов. На самом деле, решение есть – да еще какое. Практически полностью шаблонный и автоматизированный код.

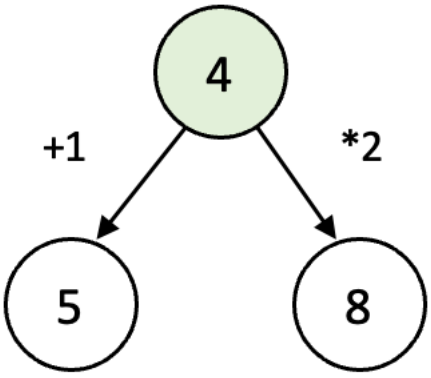
В этой статье ты на примерах разберешься в его работе, научишься легко его воссоздавать и конечно же забирать свои 3 первичных балла на любых задачах.

Для решения этой задачи нам с вами для начала нужно разобраться в теории игр и в том, как она работает наглядно.

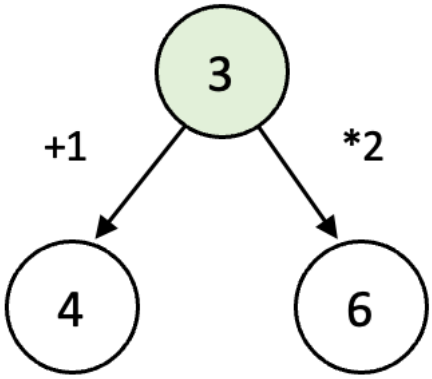
Задание 1.

Два игрока, Петя и Ваня, играют в следующую игру. Перед игроками лежит куча камней. Игроки ходят по очереди, первый ход делает Петя. За один ход игрок может добавить в кучу один камень или увеличить количество камней в куче в два раза. Для того чтобы делать ходы, у каждого игрока есть неограниченное количество камней. Игра завершается в тот момент, когда количество камней в куче становится не менее 5. Победителем считается игрок, сделавший последний ход, т.е. первым получивший такую позицию, при которой в куче будет 5 или больше камней. В начальный момент в куче было S камней; $1 \le S \le 4$. Вопрос: Кто победит, если в куче будет 4 камня?

Для того, чтобы ответить на этот вопрос удобно использовать дерево ходов, давайте изобразим дерево для первого хода Пети:



Очевидно, что для своей победы Пете достаточно выбрать любой из ходов, потому что как бы он не сходил – он получит больше или равно 5 камней. Хорошо, а кто победит, если количество камней будет равно трём?



И что же получается у Петя есть один победный и один проигрышный ход, как же быть? Да очень просто, в данных играх и моделях мы всегда принимаем, что игроки «умные» и не совершают по своей воле неосмотрительных ходов (если этого не предписано условием задачи), а значит Петя без раздумий будет ходить в кучу из 6 камней и побеждать.

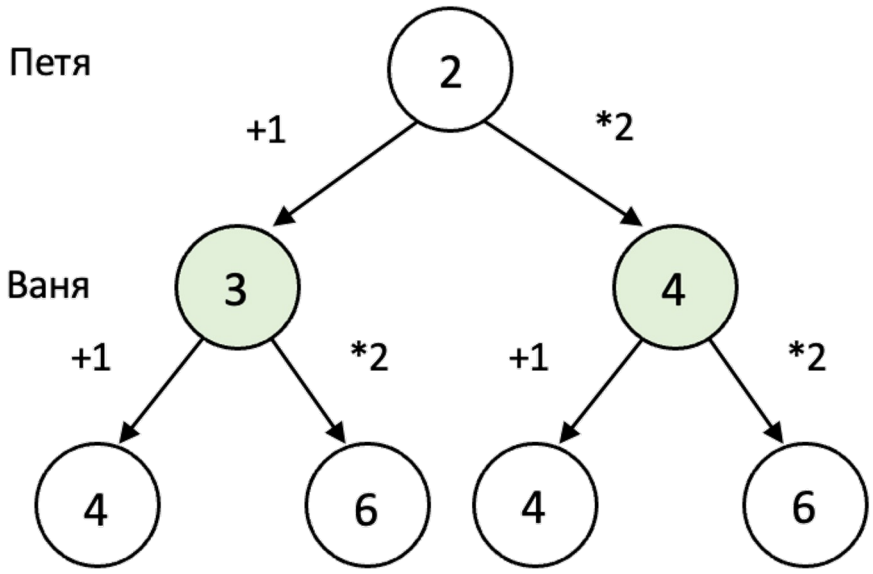
Что у нас получается, и 3, и 4 являются выигрышными позициями, потому что человек, который из них ходит (в нашем случае Петя) легко может выиграть. Давайте сформулируем понятие выигрышной стратегии:

Выигрышная позиция – эта та, из которой есть хотя бы один ход ведущий в проигрышную стратегию.

И у многих может возникнуть вопрос, как так? Выигрышная, если ведет в проигрышную? На самом деле да. Потому что главная суть теории игр – это подставить своего оппонента.

В нашем случае - 6 это проигрышная позиция, потому что если ее получит Петя, то его оппонент автоматически проиграет, соответственно именно так он и будет делать!

Ну ладно, с выигрышными стратегиями разобрались, давайте перейдем к проигрышным, но перед эти ответим на еще один вопрос: кто выиграет, если в куче будет 2 камня?



И что же мы видим? Куда бы Петя не сходил, он будет проигрывать, а знаете почему? Потому что ему сейчас приходится ходить в свои же выигрышные позиции! Если Петя сходит в 3, то Ваня спокойно оттуда победит, как это делал в прошлом Петя, аналогично и с четверкой. Получается Петя своим ходом «отдает выигрышную позицию» своему оппоненту, а значит сам он находится в проигрышной.

Давайте сформулируем понятие проигрышной стратегии:

Проигрышная позиция – эта та, из которой все ходы ведут в выигрышные позиции.

Ну и с осознанием сути уже можно переходить к коду. Будем разбирать его на примере задачи из демоверсии 2023 года.

Задание 2.

Два игрока, Петя и Ваня, играют в следующую игру. Перед игроками лежит куча камней. Игроки ходят по очереди, первый ход делает Петя. За один ход игрок может добавить в кучу один камень или увеличить количество камней в куче в два раза. Для того чтобы делать ходы, у каждого игрока есть неограниченное количество камней. Игра завершается в тот момент, когда количество камней в куче становится не менее 129. Победителем считается игрок, сделавший последний ход, т.е. первым получивший кучу из 129 или больше камней. В начальный момент в куче было S камней, $1 \le S \le 128$. Ответьте на следующие вопросы:

Даня
@ka3ak_0ff

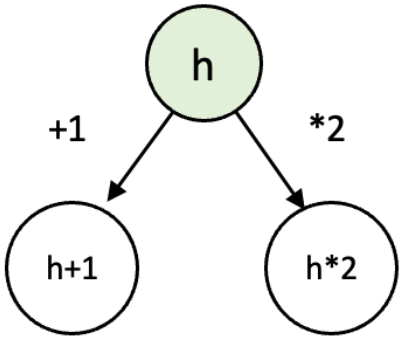
Вопрос 1. Укажите такое значение S, при котором Петя не может выиграть за один ход, но при любом ходе Пети Ваня может выиграть своим первым ходом.
Вопрос 2. Найдите два наименьших значения S, при которых у Пети есть выигрышная стратегия, причём Петя не может выиграть за один ход, но может выиграть своим вторым ходом независимо от того, как будет ходить Ваня. Найденные значения запишите в ответе в порядке возрастания.
Вопрос 3. Найдите минимальное значение S, при котором Ваня может выиграть первым или вторым ходом при любой игре Пети, но у него нет стратегии, которая позволит ему гарантированно выиграть первым ходом. Если найдено несколько значений S, в ответе запишите минимальное из них.

```
def move(h):
    return h+1, h*2
```

Теперь каждое количество камней будет исследовать и проверять по уровням, и первый уровень - это условие победы: если количество камней в куче больше либо равно 129, то значит эта позиция будет финальной и мы будем обозначать ее 'W'

```
def game(h):
    if h >= 129: return 'W'
```

Если же позиция не подходит под это условие возможно она является выигрышной за один ход. То есть нам нужно вызвать нашу функцию game для всех возможных ходов из кучи h и посмотреть если среди них есть хотя бы один выигрыш, значит эта позиция будет выигрышной за 1 ход. Наглядно это будет выглядеть так



*Если h+1 или h*2 >=129 (то есть W) то Победа Пети 1 ходом*

```
def game(h):
    if h >= 129: return 'W'
    if any(game(x) == 'W' for x in move(h)): return 'P1'
```

Для того, чтобы выиграл Ваня, Пете нужно находиться в проигрышной позиции, а чтобы он там оказался ему нужно сходить во все выигрышные.
То есть, если все позиции ведут в выигрышные(P1), то Ваня будет побеждать своим первым ходом.

```
def game(h):
    if h >= 129: return 'W'
    if any(game(x) == 'W' for x in move(h)): return 'P1'
    if all(game(x) == 'P1' for x in move(h)): return 'B1'
```

А далее все по накатанной: если хотя бы один ход попадает в проигрышную(B2), то Петя выиграет вторым ходом.

```
def game(h):
    if h >= 129: return 'W'
    if any(game(x) == 'W' for x in move(h)): return 'P1'
    if all(game(x) == 'P1' for x in move(h)): return 'B1'
    if any(game(x) == 'B1' for x in move(h)): return 'P2'
```

Для последнего условия все тоже самое, но в задании 21 нам ставят следующее условие:
«Найдите минимальное значение S, при котором Ваня может выиграть первым или вторым ходом» А раз Ваня может победить хоть первым, хоть вторым ходом, значит и в условии нам не важно в какое из них попадет наше game(x):

```
def game(h):
    if h >= 129: return 'W'
    if any(game(x) == 'W' for x in move(h)): return 'P1'
    if all(game(x) == 'P1' for x in move(h)): return 'B1'
    if any(game(x) == 'B1' for x in move(h)): return 'P2'
    if all(game(x) == 'P1' or game(x) == 'P2' for x in
move(h)): return 'B2'
```

Ну а теперь все. Пора собирать плоды нашей работы – пропишем цикл, в котором будем перебирать все исходные значения S, и будем подставлять их в нашу функцию и смотреть на результат в зависимости от условия.

Единственное, что нам нужно еще сделать это добавить в нашу функцию модуль lru_cache из стандартной библиотеки functools (разрешена на ЕГЭ), чтобы наша рекурсивная функция запоминала промежуточные результата и работа в разы быстрее.

Полный код:

Даня
@ka3ak_0ff

```python
from functools import lru_cache

def move(h):
    return h+1, h*2

@lru_cache(None)
def game(h):
    if h >= 129: return 'W'
    if any(game(x) == 'W' for x in move(h)): return 'P1'
    if all(game(x) == 'P1' for x in move(h)): return 'B1'
    if any(game(x) == 'B1' for x in move(h)): return 'P2'
    if all(game(x) == 'P1' or game(x) == 'P2' for x in
move(h)): return 'B2'

print('задание 19')
for s in range(1,100):
    if game(s) == 'B1':
        print(s, game(s))

print('задание 20')
for s in range(1,100):
    if game(s) == 'P2':
        print(s, game(s))

print('задание 21')
for s in range(1,100):
    if game(s) == 'B2':
        print(s, game(s))
```

Ответ: 64, 32 63, 62

При этом для решения задач на теорию игр можно написать и другой шаблон.

Даня
@ka3ak_0ff

Как мы проверяем является ли позиция выигрышной или нет за определенное количество ходов?

1) Если позиция превышает необходимое кол-во камней, то это позиция победы (1)
2) Если победа не было достигнута, но при этом ходы кончились, то это проигрыш (0)
3) Если ни то, ни другое, то вызывает все ходы у этой позиции и смотри куда она ведет.

```
#Передаем в функцию два числа: s – кол-во камней и m-
кол-во ходов
def game(s, m):
    #Если камни превышают условие, возвращаем равно ли m
    == 0 или игра завершилась на той же четности
    if s >= 43: return m%2==0
    #Если предыдущие условие ложно и при этом ходы
    кончились: значит ответ 0
    if m == 0: return 0
    #Формируем список всех ходов
    steps = [game(s+1,m-1), game(s+4,m-1), game(s*3,m-1)]
    #Если ход Пети (нечетное значение m), то для победы
    нужен хотя бы один ход, если Вани, то проверяем все
    ходы
    return any(steps) if m%2!=0 else all(steps)
```

Его работа будет устроена на количестве ходов. Каждый раз мы будем спрашивать функцию: «А можно ли выиграть из позиции в S камней за M ходов? И функция будет проверять:

if s >= 129: return m%2==0

То есть она смотрит, если мы получили значение s >= 129, то будем возвращать равность m==0, потому что количество ходов должно стать равным нулю, раз мы с каждым разом уменьшаем это число. Однако потенциально мы можем выиграть раньше, но ходом того же игрока, поэтому будем искать m%2

if m == 0: return 0

Даня
@ka3ak_0ff

Если же ходы закончились, а выигрыш не достигнут, то finita la commedia – возвращаем ноль.

steps = [game(s+1,m-1), game(s*2,m-1)]

Далее она формирует множество всех ходов, и на каждом шаге уменьшается исходное количество ходов на 1.

return any(steps) if m%2!=0 else all(steps)

И в заключение проверяем, если ход нечетный (то есть ходит Петя), то для его выигрыша мы проверяем наличие хотя бы одного выигрышного хода из steps, а если четный (ходит Ваня), то проверяем, чтобы все ходы были выигрышными.

Остается лишь дописать условия самих задач, и вуаля – шаблонное решение готово.

```
def game(s, m):
    if s >= 129: return m%2==0
    if m == 0: return 0
    steps = [game(s+1,m-1), game(s*2,m-1)]
    return any(steps) if m%2!=0 else all(steps)

print('19 задача:', [x for x in range(1,129) if not
game(x,0) and game(x,2)])
print('20 задача:', [x for x in range(1,129) if not
game(x,1) and game(x,3)])
print('21 задача:', [x for x in range(1,129) if not
game(x,2) and game(x,4)])
```